

Write the output of the following code snippets.

<pre>class Node { public: int data; Node* next; Node(int val) { data = val; next = nullptr; } }; void printList(Node* head) { while (head != nullptr) { cout << head->data << " "; head = head->next; } } int main() { Node* head = new Node(10); head->next = new Node(20); head->next->next = new Node(30); printList(head); return 0; }</pre>	10 20 30
<pre>class Node { public: int data; Node* prev; Node* next; Node(int val) { data = val; prev = nullptr; next = nullptr; } }; int main() { Node* first = new Node(10); Node* second = new Node(20); Node* third = new Node(30); first->prev = nullptr; first->next = second; second->prev = first; second->next = third; third->prev = second; third->next = nullptr; cout << third->prev->next->data; return 0; }</pre>	30

```

class Node {
public:
    int data;
    Node* next;

    Node(int val) {
        data = val;
        next = NULL;
    }
};

class LinkedList {
private:
    Node* head;
public:
    LinkedList() {
        head = NULL;
    }
    void insert(int val) {
        Node* newNode = new Node(val);

        if (head == NULL) {
            head = newNode;
            return;
        }

        Node* temp = head;
        while (temp->next != NULL) {
            temp = temp->next;
        }
        temp->next = newNode;
    }

    void printList() {
        Node* temp = head;
        while (temp != NULL) {
            cout << temp->data << " -> ";
            temp = temp->next;
        }
        cout << "NULL" << endl;
    }
};

int main() {
    LinkedList list;
    list.insert(10); list.insert(20);
    list.insert(30); list.insert(40);
    list.printList();

    return 0;
}

```

```

10 -> 20 ->
30 -> 40 ->
NULL

```

<pre> class Node { public: char data; Node* prev; Node* next; Node(char val) { data = val; prev = NULL; next = NULL; } }; class DoublyLinkedList { private: Node* head; Node* tail; public: DoublyLinkedList() { head = NULL; tail = NULL; } void append(char val) { Node* newNode = new Node(val); if (head == NULL) { head = tail = newNode; return; } tail->next = newNode; newNode->prev = tail; tail = newNode; } void print() { Node* temp = tail; // Start from the last node while (temp != NULL) { cout << temp->data; temp = temp->prev; } cout << endl; } }; int main() { DoublyLinkedList dll; string input = "quiz2"; // send each string char to the append function for (char ch : input) { dll.append(ch); } dll.print(); return 0; } </pre>	2ziuq
--	-------

<pre> #include <iostream> using namespace std; class Node { public: int data; Node* next; Node(int val) { data = val; next = nullptr; } }; Node* update(Node* head) { Node* prev = nullptr; Node* curr = head; while (curr != nullptr) { Node* next = curr->next; curr->next = prev; prev = curr; curr = next; } return prev; } void printList(Node* head) { while (head != nullptr) { cout << head->data << " "; head = head->next; } } int main() { Node* head = new Node(9); head->next = new Node(4); head->next->next = new Node(5); head = update(head); printList(head); return 0; } </pre>	5 4 9
--	--------------